LORA

Definition & Core Idea Behind LoRA

LoRA (Low-Rank Adaptation) is a Parameter-Efficient Fine-Tuning method that adapts large pretrained transformer models by **freezing original weights** and inserting **small trainable low-rank matrices** into important linear layers.

Core Engineering Idea

Large language models already contain strong linguistic and reasoning knowledge. Most downstream tasks only require *small behavioral adjustments*, not full retraining.

LoRA assumes that the required weight updates lie in a **low-dimensional subspace**. Instead of modifying massive matrices, LoRA learns compact update factors that *nudge* model behavior.

Three Major Problems of Full Fine-Tuning Solved by LoRA

Problem 1 — Massive GPU

Memory Usage Full fine-tuning updates billions of parameters. Optimizers must store gradients and momentum states, often requiring 3–4× model size in VRAM. This makes training large LLMs infeasible on consumer GPUs.

How LoRA Solves It: Only small low-rank matrices are trained (often <1% parameters), drastically reducing memory footprint.

Problem 2 — Slow & Expensive Training

Updating all parameters requires huge matrix multiplications and gradient updates across all layers. Training can take days or weeks.

How LoRA Solves It: Fewer trainable parameters → fewer gradient computations → faster convergence → lower compute cost.

Training Flow of LoRA (Practical Engineering View)

Step 1 — Load Pretrained Transformer

Start with a model pretrained on massive corpora. It already understands grammar, reasoning, and world knowledge.

Step 2 — Freeze Base Parameters

All original weights are locked (requires_grad=False). This preserves general intelligence and reduces memory cost.

Step 3 — Inject LoRA Modules

Selected linear layers are wrapped with LoRA adapters consisting of two low-rank matrices. These adapters form a parallel lightweight path.

Step 4 — Forward Pass

For each modified layer: Original output + LoRA correction is computed. LoRA acts as a behavior modifier.

Step 5 — Loss Computation

Predictions compared against ground truth labels using task-specific objectives.

Definition & Core Idea Behind LoRA

What This Means Practically

• Original model = frozen knowledge base • LoRA modules = lightweight skill adapters • Training cost reduces drastically • Multiple tasks can share one base model

Industry Usage

LoRA is widely used for fine-tuning large transformer models released by organizations like **Meta** and deployed through tooling ecosystems such as **Hugging Face**.

Three Major Problems of Full Fine-Tuning Solved by LoRA

Problem 3 — Storage & Deployment

Explosion Each fine-tuned task requires storing a full model copy (GBs). Maintaining multiple task models becomes impractical.

How LoRA Solves It:
Only LoRA adapters are stored (MBs). One base model can load many task-specific adapters dynamically.

Bonus — Knowledge

Preservation Full FT risks catastrophic forgetting. LoRA preserves pretrained knowledge since base weights remain frozen.

Training Flow of LoRA (Practical Engineering View)

Step 6 — Backpropagation

Gradients flow only through LoRA matrices. Frozen weights receive no updates.

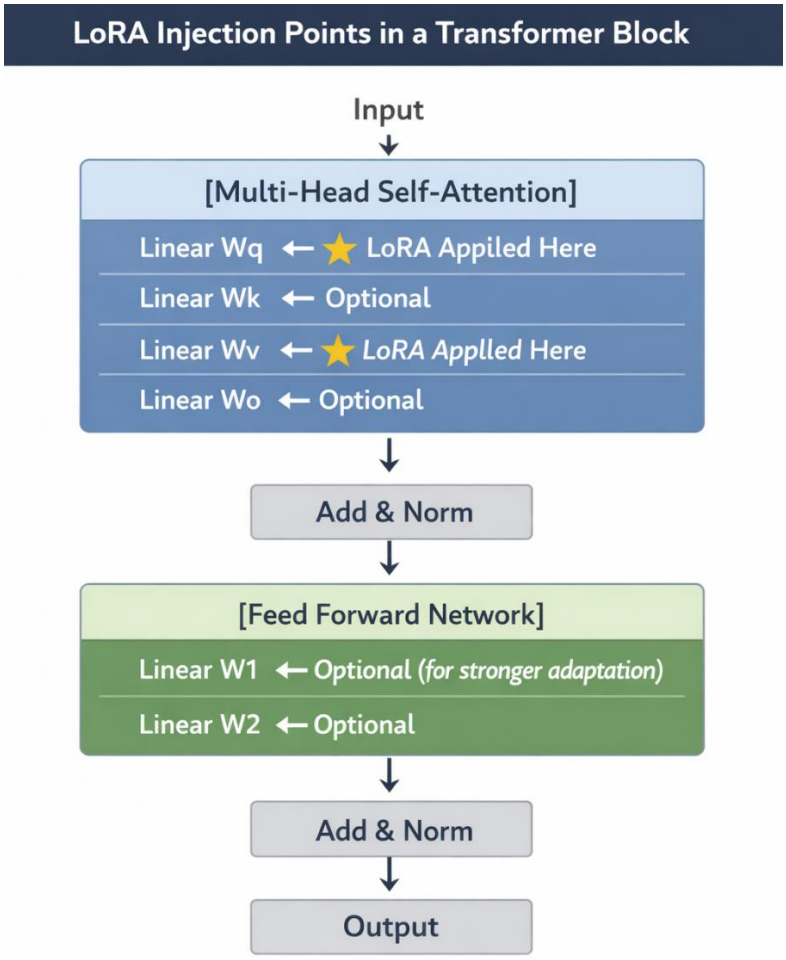
Step 7 — Parameter Update

Optimizer updates only LoRA matrices, enabling fast and memory-efficient learning.

Step 8 — Save Adapters

Only Final model consists of base model + small LoRA adapters. Adapters can be swapped for different tasks.

Exact Locations Where LoRA Is Applied in Transformers –



Mathematical Formulation (Practical Interpretation) -

Core Weight Update Rule

$$W' = W + \frac{\alpha}{r} BA$$

Practical Meaning

- W= frozen pretrained weight matrix
- B, A= small trainable matrices
- r= rank (controls adapter capacity)
- α= scaling factor (controls update strength)

LoRA does not modify pretrained weights directly. It adds a learned correction.

Forward Pass Computation

$$\text{Output} = Wx + \frac{\alpha}{r} BAx$$

$$\text{Meaning:}$$
$$\text{Original transformation} + \text{LoRA behavioral adjustment}$$

Parameter Efficiency

Full FT trains: $d \times k$parameters
LoRA trains: $r(d + k)$parameters

Since $r \ll d, k$:
Memory ↓ Compute ↓ Cost ↓

Initialization Trick

LoRA matrices initialized so that initial update ≈ 0. Model starts identical to pretrained model for stable training.

Rank Selection Strategy (Very Important in Practice)

What is Rank (r)?

Rank controls the capacity of LoRA adapters. It determines how much task-specific information adapters can store.

Low rank → small memory & fast training but limited adaptation.
High rank → better learning but increased memory & compute.

Practical Rank Guidelines

- r = 2–4 → very small tasks, classification, low-resource datasets
- r = 8 → standard NLP tasks (most common default)
- r = 16 → complex generation tasks
- r = 32+ → heavy domain shift (medical/legal LLMs)

How Engineers Choose Rank

Start small → monitor validation performance → increase rank if underfitting.

Rank Tradeoff
Rank ↑ ⇒ Quality ↑ but Memory ↑
Rank ↓ ⇒ Efficient but Risk of Underfitting ↑

Rule of Thumb

If dataset size is small, high rank leads to overfitting. Match rank to dataset complexity.

Hyperparameter Tuning for LoRA (Production Guide)

Important LoRA Hyperparameters

1.Rank (r) — Adapter capacity controller.
Too low → underfitting. Too high → memory waste & overfitting.

2.Alpha (α) — Scaling Factor
Controls strength of LoRA update.
Higher α → stronger adaptation.
Common practice: α ≈ 2r

3.Target Modules
Which layers receive LoRA.
Common: Wq, Wv
Advanced: include FFN layers for stronger adaptation.

4.Dropout
Regularizes LoRA adapters to prevent overfitting.
Typical: 0.05–0.1

5.Learning Rate
LoRA allows higher LR than full FT.
Typical: 1e-4 to 5e-4

QLORA

Definition & Core Idea Behind QLoRA	Three Major Problems Solved by QLoRA	Training Flow of QLoRA (Practical Engineering View)
QLoRA (Quantized Low-Rank Adaptation) is an advanced Parameter-Efficient Fine-Tuning technique that combines 4-bit weight quantization with LoRA adapters to enable training extremely large language models using very limited GPU memory. Core Engineering Idea	Problem 1 — GPU Memory Limits Large LLMs require enormous VRAM even for inference. Fine-tuning them normally is impossible on most hardware.	Step 1 — Load Pretrained Model Model initially stored in standard precision. Step 2 — Quantize Base Weights to 4-bit Weights compressed using advanced quantization

Definition & Core Idea Behind QLoRA

Instead of storing pretrained weights in full precision (16-bit or 32-bit), QLoRA compresses them into ultra-low precision (4-bit). These quantized weights remain frozen, while LoRA adapters learn task-specific updates.

Why It Matters

Models with tens of billions of parameters can now be fine-tuned without expensive hardware.

Industry Usage

QLoRA is widely used for efficient fine-tuning of open LLMs provided by organizations like Meta and supported in training ecosystems such as Hugging Face.

Three Major Problems Solved by QLoRA

QLoRA Solution: Quantization compresses model weights from 16-bit → 4-bit, reducing memory usage by ~75%.

Problem 2 — High Training Cost

Full-precision training requires large matrix operations and optimizer states.

QLoRA Solution: Frozen quantized weights + small LoRA adapters → very few trainable parameters → faster training.

Problem 3 — Hardware Accessibility

Previously, fine-tuning large models required multi-GPU clusters.

QLoRA Solution: Enables fine-tuning 30B–65B models on a single high-end consumer GPU.

Bonus — No Accuracy Loss

Careful quantization techniques preserve

Training Flow of QLoRA (Practical Engineering View)

schemes while preserving important information.

Step 3 — Freeze Quantized Weights

Quantized base model acts as a static knowledge backbone.

Step 4 — Inject LoRA Adapters

Low-rank matrices added to selected linear layers for learning task behavior.

Step 5 — Forward Pass

Base output computed using quantized weights, then LoRA adds behavioral correction.

Step 6 — Loss Computation

Predictions compared with labels using task objectives.

Step 7 — Backpropagation

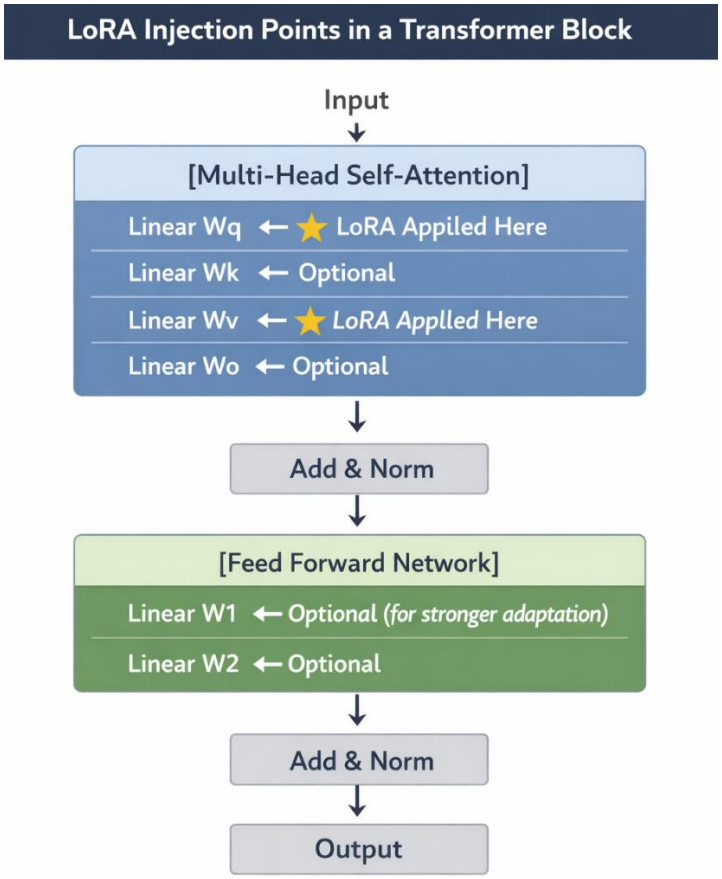
Gradients update only LoRA parameters; quantized weights remain unchanged.

Step 8 — Save LoRA Adapters

Final deployment includes quantized base model + lightweight adapters.

Definition & Core Idea Behind QLoRA	Three Major Problems Solved by QLoRA	Training Flow of QLoRA (Practical Engineering View)
	model performance close to full precision.	

Exact locations where QLORA is applied



Key Difference from LoRA
The base linear layers use quantized weights instead of full-precision weights. LoRA adapters remain full precision for stable learning.

Mathematical Formulation -

Quantized Weight Representation

Instead of storing full-precision weights, QLoRA compresses pretrained weights using low-bit quantization:

$$W \to Q(W)$$

Where:

• W= original pretrained weight matrix
• $Q(\cdot)$= quantization function (typically 4-bit NF4 quantization)
• Quantized weights are frozen and not updated during training

QLoRA Weight Update Rule

QLoRA combines quantization with LoRA adapters:

$$W' = Q(W) + \frac{\alpha}{r} BA$$

Where:

• $Q(W)$= frozen quantized pretrained weights
• $B \in \mathbb{R}^{d \times r}$= trainable matrix
• $A \in \mathbb{R}^{r \times k}$= trainable matrix
• r= rank (adapter capacity controller)
• α= scaling factor (adapter strength controller)

Only **LoRA matrices** A**and** $B$are trained.

Forward Pass Computation

During inference and training:

$$\text{Output} = Q(W)x + \frac{\alpha}{r} BAx$$

Meaning:

• Quantized base transformation
• Plus learned low-rank task adaptation

Parameter Efficiency

Let pretrained weight size be $d \times k$

• **Full Fine-Tuning trainable params**:

$$d \times k$$

• **QLoRA trainable params**:

$$r(d+k)$$

Since $r \ll d, k$:

Trainable parameters reduced by thousands of times

ADAPTERS

Definition & Core Idea Behind Adapters

Adapters are Parameter-Efficient Fine-Tuning modules where small trainable neural networks are inserted inside transformer layers to adapt pretrained models for new tasks while keeping original weights frozen.

Core Engineering Idea

Large pretrained models already contain rich language and reasoning knowledge. Instead of modifying this knowledge, adapters add small “plug-in” modules that learn how to adjust internal feature representations for new tasks. Adapters act like task-specific refinement layers that sit between major components of transformer blocks.

Practical Meaning

• Base model = frozen knowledge backbone

• Adapter modules = lightweight task learners

• Multiple adapters can be attached to one base model

• Easy to swap tasks without retraining model

Industry Usage

Adapters are widely supported in ecosystems such as Hugging Face for efficient multi-task NLP systems.

Training Flow of Adapters (Practical Engineering View)

Step 1 — Load Pretrained Transformer

Model already trained on massive corpora and contains general language intelligence.

Step 2 — Freeze Base Weights

All pretrained parameters are locked to preserve learned knowledge and reduce memory usage.

Step 3 — Insert Adapter Modules

Small bottleneck neural networks are inserted inside each transformer block.

Step 4 — Forward Pass
Hidden states flow through adapters which refine intermediate representations for the target task.

Step 5 — Loss Computation

Model predictions are compared with task labels using appropriate objective functions.

Definition & Core Idea Behind Adapters

Training Flow of Adapters (Practical Engineering View)

Step 6 — Backpropagation
Gradients update only adapter parameters; base model remains unchanged.

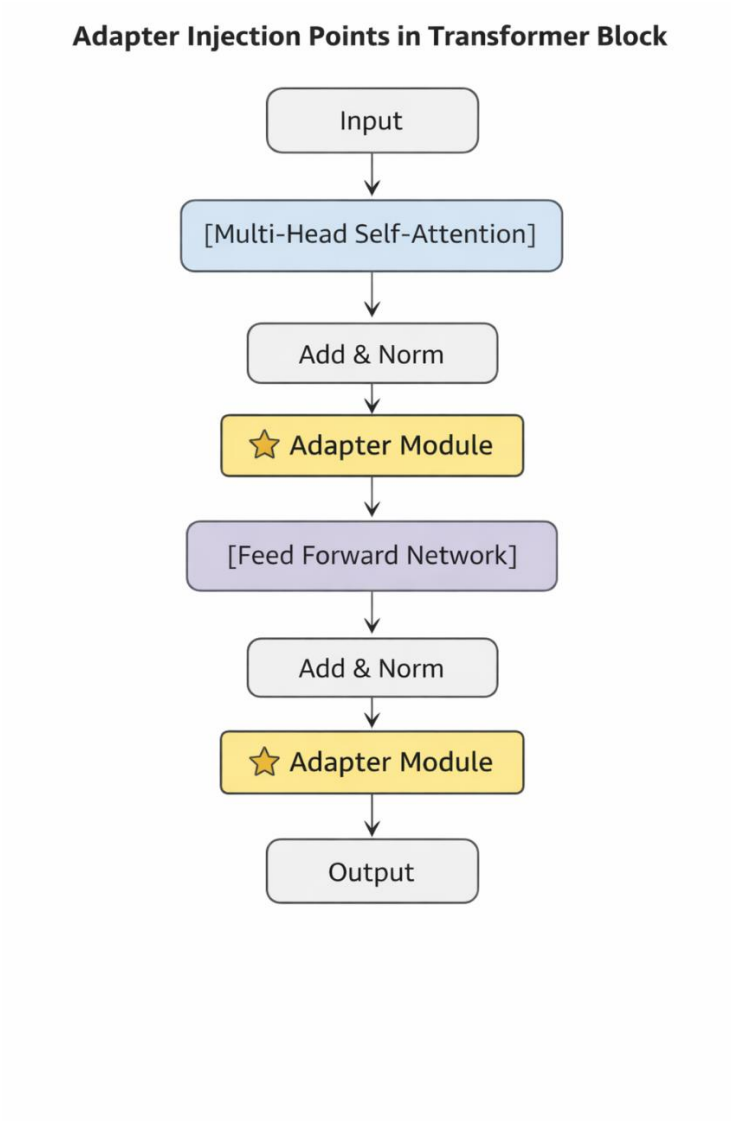
Step 7 — Parameter Update

Optimizers update the small adapter networks quickly with low compute cost.

Step 8 — Save Adapter Weights

Only adapter parameters are stored and later plugged into the base model during inference.

Exact locations where Adapter is used –



Mathematical Formulation

Adapter Internal Computation

Let the transformer hidden state be:

$$h \in \mathbb{R}^d$$

Adapters follow a bottleneck architecture:

Down Projection → Non-linearity → Up Projection

$$\text{Adapter}(h) = W_{\text{up}}\ \sigma(W_{\text{down}}\ h)$$

Where:

- $W_{\text{down}} \in \mathbb{R}^{d \times r} \rightarrow$ reduces dimensionality
- $W_{\text{up}} \in \mathbb{R}^{r \times d} \rightarrow$ restores dimensionality

• $r \ll d \rightarrow$ bottleneck dimension (controls adapter capacity)
• $\sigma \rightarrow$ activation function (ReLU / GELU)

Residual Connection (Final Output)

Adapters are added via a residual skip connection:

$$h' = h + W_{\text{up}}\ \sigma(W_{\text{down}}\ h)$$

Practical Interpretation

• Original representation $h$is preserved
• Adapter learns small task-specific corrections
• Residual structure ensures stable training
• Only adapter parameters are updated during fine-tuning

DORA

Definition & Core Idea Behind DoRA

DoRA (Weight-Decomposed Low-Rank Adaptation) is an improved version of LoRA that decomposes pretrained weight matrices into magnitude and direction components, allowing fine-tuning to modify weight *direction* efficiently while preserving weight *scale*.

Core Engineering Idea LoRA assumes weight updates lie in a low-rank space but treats weights as a single entity. However, neural network weights contain:

• **Magnitude (norm)** → strength of features

• **Direction** → information pattern

DoRA fine-tunes direction separately while keeping magnitude

Problems in LoRA Solved by DoRA

Problem 1 — Limited Expressiveness of LoRA

LoRA's low-rank updates sometimes lack capacity for complex tasks.

DoRA Solution: Separating magnitude and direction allows richer representation updates.

Problem 2 — Training Instability

LoRA may produce unstable updates when scaling factors interact

Training Flow of DoRA (Practical Engineering View)

Step 1 — Load Pretrained Transformer

Model contains pretrained weights W.

Step 2 — Decompose Weights

Each weight matrix decomposed into: Magnitude (scale) + Direction (unit vector).

Step 3 — Freeze Magnitude

Component Weight norms remain unchanged to

Definition & Core Idea Behind DoRA

stable, resulting in better adaptation and stability.

Practical Meaning

• More expressive than LoRA • Better learning capacity • More stable optimization • Better performance on complex tasks

Intuition

Instead of changing “how strong” features are, DoRA focuses on “how features are oriented” in vector space.

Problems in LoRA Solved by DoRA

poorly with pretrained weights.

DoRA Solution: Magnitude preservation stabilizes gradient flow.

Problem 3 — Suboptimal Weight Adaptation

LoRA modifies weights additively, which may distort pretrained structures.

DoRA Solution: Directional updates preserve pretrained knowledge geometry.

Training Flow of DoRA (Practical Engineering View)

preserve feature strength.

Step 4 — Apply Low-Rank Adaptation to Direction

LoRA-like low-rank matrices modify directional component.

Step 5 — Forward Pass

Updated direction recombined with original magnitude.

Step 6 — Loss Computation

Predictions compared with labels.

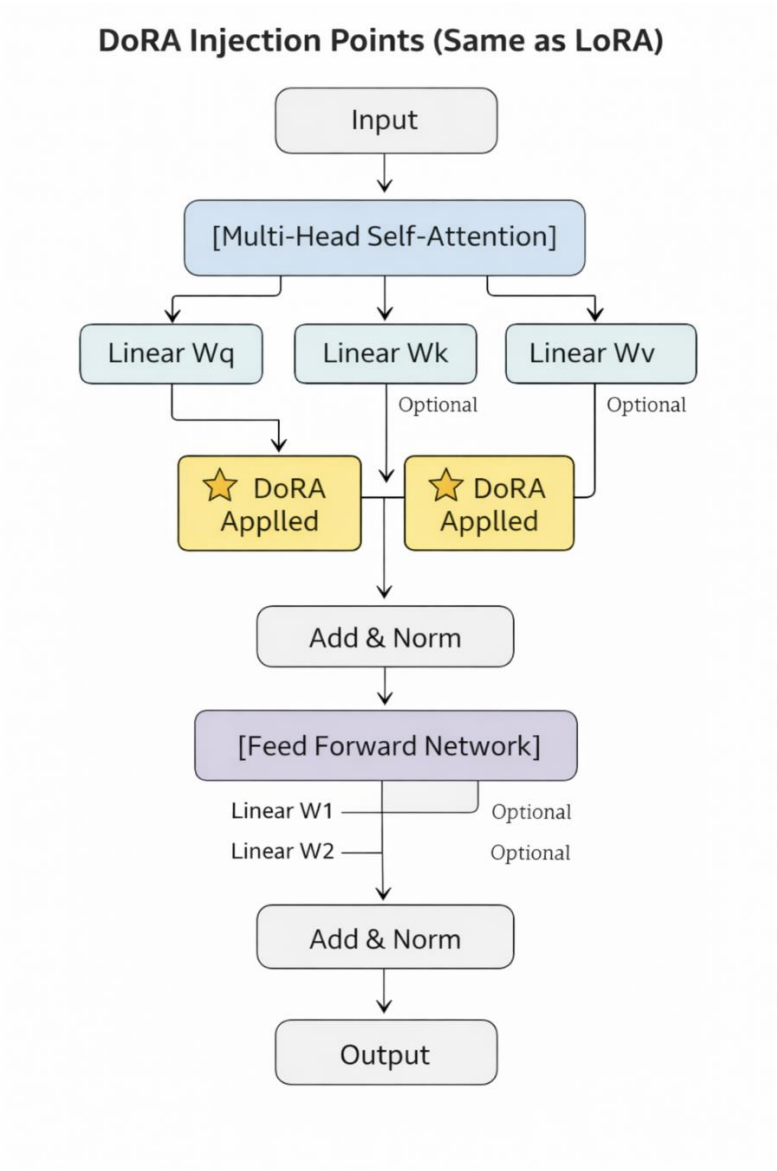
Step 7 — Backpropagation

Gradients update only direction adapters.

Step 8 — Save DoRA Adapters

Adapters stored similar to LoRA modules.

WHEN TO USE DORA –



Mathematical Formulation –

Weight Decomposition

Each pretrained weight matrix is decomposed into magnitude and direction:

$$W = m \cdot V$$

Where:

• m= magnitude (scalar norm of weight matrix)
• V= direction component (unit-norm matrix capturing structural information)

Direction Update with Low-Rank Adaptation

DoRA applies low-rank adaptation to the directional component:

$$V' = V + \frac{\alpha}{r} BA$$

Where:

- $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$ are trainable low-rank matrices
- r = rank controlling adapter capacity
- α = scaling factor controlling update strength

Final Reconstructed Weight

After updating the direction, the weight matrix is reconstructed:

$$W' = m \cdot V'$$

Forward Pass Computation

$$\text{Output} = (m \cdot V')x$$

Meaning:

- Magnitude of pretrained weights is preserved
- Direction is adapted to learn task-specific behaviour.

INSTRUCTION BASED FINE TUNING

Definition & Core Idea Behind Instruction Fine-Tuning	Problems Solved by Instruction Fine-Tuning	Training Flow (Practical Engineering View)	Difference Between Instruction Fine-Tuning and PEFT
Instruction-Based Fine-Tuning (IFT) is a supervised training approach where a pretrained language model is trained on instruction–response pairs so it learns to	Problem 1 — Poor Instruction Following Pretrained models generate text continuations	Step 1 — Start with Pretrained LLM Model already understands language and reasoning.	Core Difference: Purpose **Instruction FT:** Teaches model *how to behave* and follow tasks.

Definition & Core Idea Behind Instruction Fine-Tuning

follow human commands and generate helpful task-oriented outputs.

Core Engineering Idea

Pretrained LLMs are strong text predictors but not reliable task solvers. Instruction tuning teaches the model how to convert user requests into useful responses. Instead of continuing text, the model learns task understanding → structured response generation.

Practical Meaning

• Model learns assistant-like behavior

• Understands task intent

• Produces formatted, relevant answers

• Enables conversational AI systems

Industry Usage

This method is central to assistant-style systems such as ChatGPT developed

Problems Solved by Instruction Fine-Tuning

but may ignore user intent.

IFT Solution: Teaches explicit mapping from instructions to task-appropriate responses.

Problem 2 — Irrelevant & Unstructured Outputs

Raw LLMs may ramble or produce unformatted answers.

IFT Solution: Models learn structured answering patterns.

Problem 3 — No Conversational Behavior

Pretrained models lack multi-turn dialogue skills.

IFT Solution: Training on dialogue-style instructions

Training Flow (Practical Engineering View)

Step 2 — Build Instruction Dataset

Human-written instructions paired with ideal responses.

Step 3 — Format Data Instruction → Response pairs structured as supervised examples.

Step 4 — Supervised Fine-Tuning

Teacher forcing used so model predicts correct response tokens.

Step 5 — Loss Computation

Cross-entropy loss between predicted and ground-truth tokens.

Step 6 — Backpropagation

Model weights updated to improve instruction-following ability.

Difference Between Instruction Fine-Tuning and PEFT

PEFT (LoRA/Adapters): Teaches model *new knowledge or domain skills* efficiently.

Parameter Updates IFT

(Full FT): Updates most or all model weights.

PEFT: Updates only small added modules.

Hardware Cost

IFT: Expensive, needs large GPUs.

PEFT: Lightweight and memory-efficient.

Dataset Type

IFT: Instruction–response datasets.

PEFT: Can use any task-specific labeled dataset.

Architectural Changes

IFT: No new modules added. PEFT: Adds adapters or low-rank modules.

Definition & Core Idea Behind Instruction Fine-Tuning	Problems Solved by Instruction Fine-Tuning	Training Flow (Practical Engineering View)	Difference Between Instruction Fine-Tuning and PEFT
by OpenAI and Gemini from Google.	enables chat capability. Bonus — Multi-Task Learning Instruction datasets contain diverse tasks → single model learns many skills.	### □ Step 7 — Assistant Behavior Emerges Model learns helpful answering patterns.	When Used Together Modern systems often combine both → Instruction Tuning with LoRA for efficiency.

Where Changes Occur in the Transformer

Parameter Updates in Transformer

Instruction tuning usually updates **all major components** of the transformer:

✓ Attention layers
✓ Feed-forward networks
✓ Layer norms
✓ Output projection layers

Key Point
No architectural changes are introduced. Only the weights are updated.

Two Common Approaches
🔘 Full Fine-Tuning → all parameters updated
□ PEFT + Instruction Tuning → LoRA/Adapters used for efficiency